

Projekt Big Data

Składowanie danych w systemach Big Data

Semestr 7

Autorzy:

Jan Cwalina, Mikołaj Guzik

Wydział Matematyki i Nauk Informacyjnych
Politechnika Warszawska
15 stycznia 2026

Spis treści

1	Cel projektu	2
2	Uzasadnienie biznesowe	2
3	Źródła danych	2
3.1	Opis danych	3
4	Architektura	4
4.1	Plan pozyskiwania danych	5
4.2	Technologie	5
4.3	Przepływ danych	6
4.4	Diagram architektury	7
4.5	Struktura katalogów projektu	7
4.5.1	Lokalny system plików maszyny wirtualnej	7
4.5.2	Rozproszony system plików HDFS (docelowe miejsce przechowywania danych)	8
4.5.3	Struktura w Apache HBase	8
5	Apache NiFi - ingest i preprocessing danych	9
5.0.1	Tabele wymiarowe używane w projekcie	9
5.1	Apache NiFi — przepływ ingestu i preprocessing danych Airbnb	9
5.1.1	Struktura danych ładujących w Hive i Kafka	13
5.2	Ingest i preprocessing danych policyjnych NYC	15
5.2.1	Struktura danych NYPD ładujących w Hive i Kafka	15
5.3	Ingest i preprocessing danych policyjnych UK	16
5.3.1	Struktura danych UKPD ładujących w Hive i Kafka	16
5.4	Rola Apache NiFi w architekturze	16
6	Apache Spark - Przetwarzanie batchowe (Spark Batch)	17
6.1	Wspólny klucz przestrzenny (geohash)	17
6.2	Notebook 1: Czyszczenie danych Airbnb	17
6.3	Notebook 2: Unifikacja danych policyjnych	18
6.4	Notebook 3: Join danych Airbnb z przestępczością	18
6.5	Notebook 4: Statystyki dzielnic i porównania miast	19
6.6	Podsumowanie etapu batchowego	19
7	Apache Hive (SQL interface)	20
7.1	Warstwa tabel zewnętrznych	20
7.2	Pytania analityczne i wyniki	20
7.3	Podsumowanie	22
8	Wizualizacja wyników	23
8.1	Heatmapa intensywności przestępczości i ofert Airbnb - Londyn	23
8.2	Heatmapa intensywności przestępczości i ofert Airbnb - Nowy Jork	23
8.3	Choropleth: średnia liczba przestępstw w pobliżu ofert Airbnb (agregacja dzielnicowa)	24
8.4	Podsumowanie	26
9	Testy	27
9.1	Testy funkcjonalne pipeline'u Spark	27
9.2	Testy funkcjonalne Apache Hive (tabele zewnętrzne)	28
9.3	Testy funkcjonalne Apache HBase (warstwa NoSQL)	28

1 Cel projektu

Celem projektu jest stworzenie procesu przetwarzania i analizy danych dla dwóch miast: Londynu i Nowego Jorku. Projekt zakłada pozyskanie oraz integrację danych z trzech niezależnych źródeł: platformy **Inside Airbnb** oraz serwisów **data.police.uk** i **NYC Open Data**.

Głównym założeniem jest zbudowanie środowiska typu Big Data, które umożliwi gromadzenie, przetwarzanie i wizualizację danych z wykorzystaniem narzędzi takich jak Apache NiFi, Hadoop, Hive i Spark. W efekcie system pozwoli badać zależności pomiędzy aktywnością wynajmu krótkoterminowego a poziomem przestępczości w poszczególnych dzielnicach wybranych miast.

Projekt ma charakter analityczny i może stanowić podstawę dla decyzji podejmowanych przez administrację miejską, inwestorów lub platformy turystyczne. Analiza pozwoli zidentyfikować obszary, w których wysoka intensywność wynajmu pokrywa się z większą liczbą zgłoszonych przestępstw, co może być wykorzystane do oceny wpływu turystyki i krótkoterminowego wynajmu na bezpieczeństwo publiczne. Dodatkowo, zbudowana architektura przetwarzania danych może być rozwijana w kierunku cyklicznego monitorowania trendów miejskich w czasie rzeczywistym.

2 Uzasadnienie biznesowe

Projekt ma charakter analityczny i może być wykorzystany przez różne grupy interesariuszy, dla których istotne jest zrozumienie zależności pomiędzy intensywnością wynajmu krótkoterminowego a poziomem bezpieczeństwa publicznego.

- **Władze miejskie i samorządy lokalne** – mogą wykorzystać wyniki analizy do lepszego planowania polityki mieszkaniowej, turystycznej i bezpieczeństwa. Dane pozwolą identyfikować obszary o dużej koncentracji wynajmów Airbnb oraz zwiększonym ryzyku występowania przestępstw.
- **Służby porządkowe i policja** – analiza przestrzenna może pomóc w efektywniejszym rozmieszczaniu patroli i alokacji zasobów w dzielnicach, gdzie obserwowany jest wzrost aktywności turystycznej i przestępczości.
- **Firmy i inwestorzy z sektora nieruchomości** – uzyskane dane mogą wspierać decyzje dotyczące inwestycji w wynajem krótkoterminowy lub długoterminowy, uwzględniając poziom bezpieczeństwa w danym obszarze.
- **Platformy turystyczne i właściciele nieruchomości** – mogą analizować, jak bezpieczeństwo wpływa na liczbę rezerwacji i ceny wynajmu w poszczególnych dzielnicach.

W ujęciu szerszym projekt ma także znaczenie edukacyjne i badawcze. Integracja otwartych danych publicznych z różnych źródeł pozwala zrozumieć, jak technologie Big Data mogą wspierać analizy miejskie, zarządzanie przestrzenią i rozwój miast.

3 Źródła danych

Inside Airbnb

- **Adres:** <https://insideairbnb.com/get-the-data/>
- **Zakres:** dane o ofertach wynajmu w Londynie (snapshot z 10 czerwca 2025) i Nowego Jorku (snapshot z 1 listopada 2025)
- **Format:** CSV (**listings**) + GeoJSON (**neighbourhoods**)
- **Główne atrybuty:** latitude, longitude, price, neighbourhood, room_type, number_of_reviews, itp.
- **Częstotliwość aktualizacji:** co 3–4 miesiące (dane publikowane w formie snapshotów obejmujących stan na konkretny dzień)
- **Cel użycia:** analiza rozmieszczenia ofert Airbnb i porównanie z poziomem przestępczości

data.police.uk

- **Adres:** <https://data.police.uk/data/>
- **Zakres:** dane o przestępczości Londynie z czerwca 2025
Wybrane jednostki: Metropolitan Police Service (Greater London).
- **Format:** CSV
- **Główne atrybuty:** Crime type, Month, Latitude, Longitude, Location, LSOA name
- **Częstotliwość aktualizacji:** co miesiąc, z opóźnieniem około 2 miesięcy względem dzisiejszej daty
- **Cel użycia:** analiza rozmieszczenia i rodzaju przestępstw w tych samych obszarach co oferty Airbnb

NYC Open Data

- **Adres:** <https://data.cityofnewyork.us/Public-Safety/NYPD-Arrest-Data-Year-to-Date->
- **Zakres:** dane o przestępczości aresztowaniach w NYC w 2025 roku.
- **Format:** CSV/JSON
- **Główne atrybuty:** ARREST_DATE, PD_DESC, Latitude, Longitude, Location, PREP_SEX, PREP_RACE
- **Częstotliwość aktualizacji:** co dzień
- **Cel użycia:** analiza rozmieszczenia i rodzaju przestępstw w tych samych obszarach co oferty Airbnb

3.1 Opis danych

Inside Airbnb

Zbiory `listings.csv` zawierają dane opisujące wszystkie aktywne oferty wynajmu w serwisie Airbnb dla miasta Londyn. Każdy wiersz odpowiada jednej ofercie wynajmu.

- **Liczba rekordów:** około 96 tys. (Londyn)
- **Liczba kolumn:** 79
- **Najważniejsze atrybuty:**
 - `id` – unikalny identyfikator oferty,
 - `name` – nazwa oferty,
 - `host_id`, `host_name` – dane identyfikujące gospodarza,
 - `neighbourhood`, `neighbourhood_cleansed` – nazwa dzielnicy lub obszaru,
 - `latitude`, `longitude` – współrzędne geograficzne oferty,
 - `room_type` – typ zakwaterowania (np. Entire home/apt, Private room),
 - `price` – cena za noc (GBP),
 - `number_of_reviews` – liczba opinii użytkowników,
 - `availability_eoy` – liczba dni, w których oferta była dostępna w ciągu roku.
- **Plik GeoJSON:** `neighbourhoods.geojson` – zawiera granice administracyjne dzielnic obu miast wykorzystywane do agregacji i wizualizacji przestrzennej.

data.police.uk

Zbiory danych z portalu `data.police.uk` zawierają informacje o przestępstwach zgłoszonych w czerwcu 2025 roku na obszarze działania dwóch jednostek policyjnych: Metropolitan Police Service (Greater London) oraz Avon and Somerset Constabulary.

Każdy wiersz w pliku CSV reprezentuje jedno zdarzenie kryminalne z przypisaną kategorią i lokalizacją.

- **Liczba rekordów:** około 100 tys. (Londyn).
- **Liczba kolumn:** 12
- **Najważniejsze atrybuty:**
 - `Crime ID` – identyfikator zdarzenia,
 - `Month` – miesiąc zgłoszenia,
 - `Reported by` – jednostka policyjna rejestrująca zdarzenie,
 - `Crime type` – kategoria przestępstwa (np. burglary, robbery, vehicle crime),
 - `Last outcome category` – wynik postępowania (np. investigation complete, no suspect identified),
 - `Latitude`, `Longitude` – przybliżone współrzędne miejsca zdarzenia,
 - `Location` – opis lokalizacji (np. Near High Street),
 - `LSOA name` – jednostka statystyczna (Lower Layer Super Output Area).

NYC Open Data

Zbiory danych z portalu `data.police.uk` zawierają informacje o aresztowaniach dokonanych w roku 2025.

Każdy wiersz w pliku CSV reprezentuje jedno aresztowanie z przypisaną kategorią i lokalizacją.

- **Liczba rekordów:** około 212 tys.
- **Liczba kolumn:** 19
- **Najważniejsze atrybuty:**
 - `ARREST_KEY` – identyfikator zdarzenia,
 - `ARREST_DATE` – dokładna data aresztowania,
 - `PD_DESC` – opis klasy zdarzenia,
 - `OFNS_CODE` – opis klasyfikacji wewnętrznej odpowiadający kodowi KY (bardziej ogólny niż PD),
 - `ARREST_BORO` – obszar zdarzenia (B(Bronx), S(Staten Island), K(Brooklyn), M(Manhattan), Q(Queens)),
 - `Latitude`, `Longitude` – przybliżone współrzędne miejsca zdarzenia,
 - `Location` – opis lokalizacji (np. Near High Street),
 - `PERP_RACE` – rasa aresztowanego.

4 Architektura

Celem architektury jest zbudowanie hybrydowego pipeline’u przetwarzania danych, łączącego przetwarzanie wsadowe (*batch*) z symulowanym strumieniowym (*streaming*), który integruje dane z trzech niezależnych źródeł (Inside Airbnb, data.police.uk oraz NYC Open Data) i przygotowuje je do analiz przestrzennych dla Londynu oraz porównań z Nowym Jorkiem. Ze względu na zakres projektu stosujemy rozwiązanie jedno-węzłowe (lokalne), zachowując koncepcję typowej architektury Big Data (warstwy danych, rozdział ingestu, składowania i przetwarzania).

4.1 Plan pozyskiwania danych

W projekcie przyjęto hybrydowy model pozyskiwania danych, obejmujący dane z grudnia 2025 roku. Celem jest analiza porównawcza rozmieszczenia ofert wynajmu krótkoterminowego oraz aktywności przestępczej w tym samym okresie dla Londynu z odniesieniem do aktywności policyjnej w Nowym Jorku. Dane policyjne będą symulowane jako strumieniowe na podstawie dat zgłoszeń.

Dane planuje się pobrać z trzech niezależnych źródeł publicznych:

- z serwisu Inside Airbnb, który udostępnia szczegółowe informacje o ofertach wynajmu krótkoterminowego. Dla obu miast zostaną wykorzystane pliki `listings.csv` oraz `neighbourhoods.geojson` w wersji z 6 grudnia 2025 roku, zawierające dane o lokalizacji, cenie, typie zakwaterowania i liczbie recenzji, itp.,
- z portalu `data.police.uk`, który publikuje dane o przestępczości na poziomie ulicy. Dla Londynu zostanie użyta jednostka Metropolitan Police Service, a dla. Planowane jest pobranie danych z grudnia 2025 roku w formacie CSV (pliki ZIP rozpakowane do lokalnego katalogu projektu),
- z serwisu NYC Open Data, udostępniającego dane o aktywności policyjnej poprzez API Socrata (endpoint: `https://data.cityofnewyork.us/resource/uip8-fykc.json`). Dane będą pobierane w formacie JSON i konwertowane do formatu Avro, obejmując aresztowania z grudnia 2025 roku, z kolumnami takimi jak `data`, `aresztowania`, `opis przestępstwa` i `lokalizacja`.

Dane z Inside Airbnb zostaną zapisane bezpośrednio w HDFS w warstwie `raw` data. Dane policyjne (z `data.police.uk` i NYC) będą symulowane jako strumieniowe poprzez Apache NiFi, który odczytuje dane z plików lokalnych i kieruje je do Apache Kafka w oparciu o daty zgłoszeń. Na tym etapie nie przewiduje się cyklicznego odświeżania danych, ponieważ celem projektu jest jednorazowa analiza przekrojowa z symulacją strumienia.

W ramach weryfikacji poprawności planowane jest sprawdzenie plików pod kątem zgodności formatu (rozszerzenia `.csv`, `.geojson`, `.json`) oraz liczby rekordów większej od zera. W przypadku wykrycia błędów (np. pustych archiwów ZIP lub nieudanych zapytań API) pliki będą wykluczane z dalszego przetwarzania.

Celem etapu jest przygotowanie kompletnych i poprawnych danych źródłowych, które posłużą do dalszych etapów przetwarzania i analizy w środowisku Big Data.

4.2 Technologie

- **Apache NiFi (ingest)** – automatyzuje proces ingestu danych: pobiera dane Airbnb (CSV, GeoJSON) oraz dane policyjne (CSV z `data.police.uk`, przetworzony Avro z NYC Open Data), wykonuje walidację i normalizację formatów, a następnie kieruje dane batchowe (Airbnb) bezpośrednio do HDFS oraz symuluje strumieniowe przetwarzanie danych policyjnych poprzez routing do Apache Kafka.
- **Apache Kafka (streaming broker)** – pełni rolę brokera komunikatów dla symulowanego strumienia danych policyjnych (topic: `police-stream`), umożliwiając asynchroniczne przetwarzanie zdarzeń w czasie rzeczywistym. Kafka odbiera dane z NiFi i dostarcza je do Apache Spark Structured Streaming.
- **HDFS / Hadoop (batch storage)** – stanowi główne repozytorium dla danych batchowych, z uporządkowaną strukturą katalogów `data/` podzieloną na poziomy: `raw` (dane w oryginalnej formie z Airbnb), `processed` (po wstępnym przetworzeniu i czyszczeniu) oraz `analytical` (gotowe do analiz). Dane są składowane w formacie Parquet dla efektywności przetwarzania.
- **Apache HBase (NoSQL storage)** – przechowuje ujednolicone dane policyjne pochodzące ze strumienia Kafka. HBase pozwala na szybki dostęp do danych o przestępczości według klucza (`data + geohash`) i elastyczne dodawanie kolumn dla różnych źródeł (UK i NYC), pomimo różnic w schematach danych źródłowych.
- **Apache Spark (unified processing engine)** – odpowiada za całościowe przetwarzanie danych: *Spark Streaming* konsumuje dane policyjne z Kafka, ujednolica różne schematy (UK vs NYC) i zapisuje do HBase; *Spark Batch* przetwarza dane Airbnb z HDFS, wykonuje czyszczenie, joiny przestrzenne (z danymi policyjnymi z HBase), agregacje na poziomie dzielnic/LSOA i zapisuje wyniki do HDFS w warstwie `processed` i `analytical`.

- **Apache Hive (SQL interface)** – udostępnia warstwę SQL nad danymi składowanymi w HDFS, umożliwiając tworzenie zewnętrznych tabel nad danymi w Parquet oraz integrację z danymi w HBase poprzez Hive-HBase connector. Pozwala na wykonywanie zapytań analitycznych SQL i łatwe połączenie z narzędziami Business Intelligence do raportowania.
- **Warstwa wizualizacji** – notebook (Python) lub narzędzie BI umożliwia tworzenie map oraz wykresów porównawczych pokazujących kluczowe zależności w danych, w tym porównania z NYC.

4.3 Przepływ danych

1. Ingest (*NiFi*):

- Dane Airbnb: pobranie plików `listings.csv` i `neighbourhoods.geojson`, walidacja i bezpośredni zapis do HDFS w warstwie *raw*.
- Dane policyjne NYC: pobranie JSON via API Socrata, konwersja do formatu Avro poprzez procesor QueryRecord.
- Dane policyjne UK: odczyt z lokalnych plików CSV (rozpakowane ZIP z `data.police.uk`).
- Symulacja strumienia: routing danych policyjnych (NYC i UK) do Kafka topic `police-stream` w oparciu o daty zgłoszeń.

2. Przetwarzanie strumieniowe (*Spark Streaming + HBase*):

- Spark Structured Streaming konsumuje dane z Kafka topic `police-stream`.
- Ujednolicenie schematów: transformacja różnych kolumn z UK i NYC do wspólnego formatu.
- Czyszczenie i walidacja: filtrowanie nieprawidłowych współrzędnych, standaryzacja typów przestępstw.
- Zapis do HBase: dane policyjne zapisywane z kluczem złożonym (`data + geohash`) dla szybkiego dostępu przestrzennego.

3. Przetwarzanie batchowe (*Spark Batch*):

- Odczyt danych Airbnb z HDFS warstwy *raw*.
- Czyszczenie: usunięcie duplikatów, parsowanie cen, standaryzacja nazw dzielnic/LSOA, kontrola zakresów `lat/lon`.
- Odczyt danych policyjnych z HBase.
- Joiny przestrzenne: przypisanie ofert Airbnb i przestępstw do poligonów (`neighbourhoods/LSOA`) lub siatki geohash.
- Zapis danych przetworzonych do HDFS w warstwie *processed* (format Parquet).

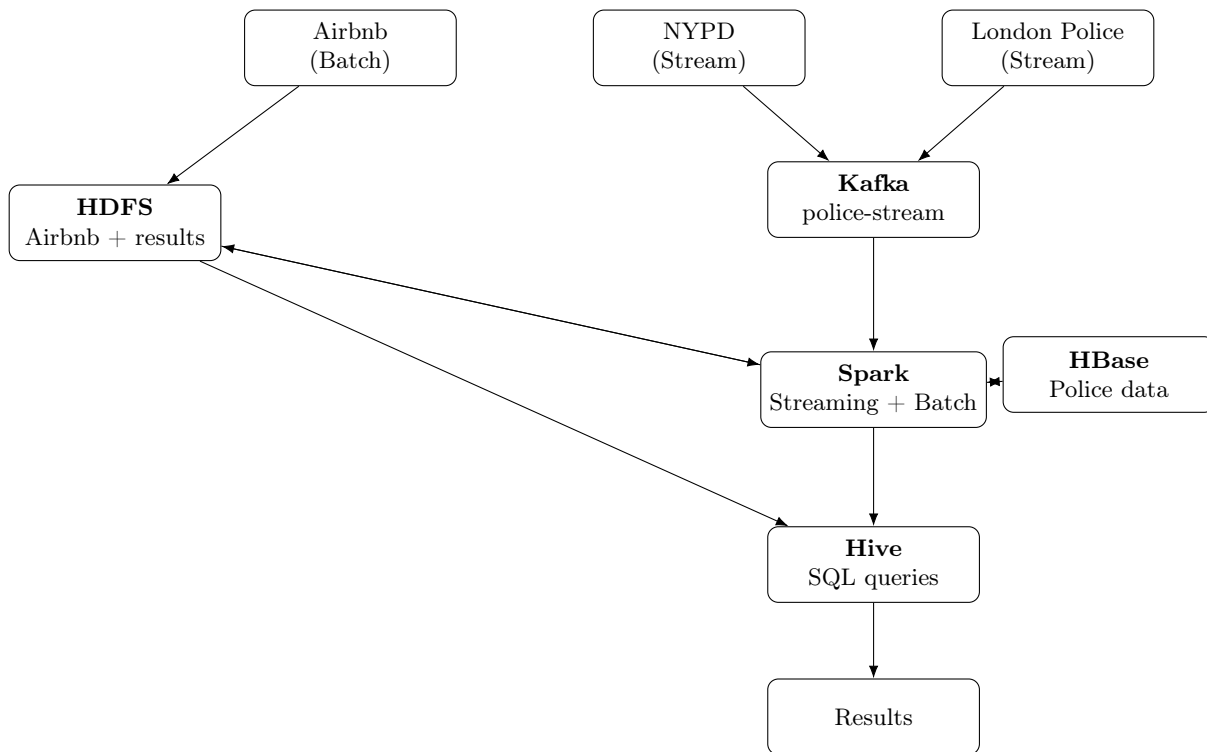
4. Agregacje i analityka (*Spark*):

- Obliczanie metryk per dzielnica: liczba ofert, mediana ceny, liczba przestępstw, wskaźniki bezpieczeństwa.
- Porównania międzymiastowe (London vs NYC).
- Zapis wyników do HDFS w warstwie *analytical* (format Parquet).

5. Zapytania SQL i raportowanie (*Hive*):

- Utworzenie zewnętrznych tabel Hive nad danymi w HDFS (*processed, analytical*).
- Integracja Hive-HBase dla dostępu SQL do danych policyjnych.
- Wykonywanie zapytań analitycznych i eksport wyników do wizualizacji.

4.4 Diagram architektury



4.5 Struktura katalogów projektu

Ze względu na jednowęzłowy charakter środowiska oraz konieczność zapewnienia powtarzalności, bezpieczeństwa danych i łatwości współpracy zdalnej, przyjęto ścisły podział katalogów zarówno w lokalnym systemie plików maszyny wirtualnej, jak i w rozproszonym systemie plików HDFS oraz bazie HBase.

4.5.1 Lokalny system plików maszyny wirtualnej

Wszystkie duże pliki danych są świadomie wyłączone z repozytorium Git (poprzez wpis w pliku `.gitignore`), dzięki czemu repozytorium pozostaje lekkie, a partner może je sklonować bez problemów.

```
/home/<user>/bigdata-project/  
data/                                # ← cały folder w .gitignore  
  raw/                               # tylko do ręcznych testów / backupu  
  temp/                             # tymczasowe pliki NiFi (automatycznie czyszczone)  
scripts/                             # skrypty pomocnicze  
  download_airbnb.sh  
  download_police_uk.py  
  download_nyc_api.py  
nifi-templates/                      # eksportowane szablony flow  
spark-jobs/                          # .py lub .scala z zadaniami Spark  
hive-sql/                           # pliki .hql  
notebooks/                          # Jupyter / Zeppelin  
docs/                               # dokumentacja projektu  
README.md, environment.yml, ...
```

Uwaga: w docelowym rozwiązaniu folder `data/raw/` nie jest używany w codziennej pracy – wszystkie surowe pliki trafiają bezpośrednio do HDFS.

4.5.2 Rozproszony system plików HDFS (docelowe miejsce przechowywania danych)

Wszystkie surowe i przetworzone zbiory danych przechowywane są wyłącznie w HDFS, co zapewnia replikację, bezpieczeństwo i najwydajniejszy dostęp dla Spark i Hive.

```
hdfs dfs -ls -R /user/<user>/bigdata
```

```
/user/<user>/bigdata/  
raw/  
  airbnb/  
  police/  
    police_nyc/  
    police_uk/  
processed/  
  airbnb/  
  police/  
    police_nyc/  
    police_uk/  
analytical/  
dimensions/
```

Katalogi tworzone są raz na początku projektu poleceniem:

```
hdfs dfs -mkdir -p /user/$USER/bigdata/{raw/{airbnb/,  
  \police_uk/,police_nyc},processed/{airbnb_clean,police_events,geo},\  
analytical/{agg_neighbourhood,agg_geohash,reports}}  
hdfs dfs -chmod -R 755 /user/$USER/bigdata
```

4.5.3 Struktura w Apache HBase

Namespace: bigdata

Tabela: bigdata:police_events

Column families:

```
cf_common : pola wspólne dla UK i NYC (crime_type, latitude, longitude, timestamp)  
cf_uk      : pola specyficzne dla data.police.uk (lsoa_code, reported_by, falls_within, ...)  
cf_nyc     : pola specyficzne dla NYC Open Data (pd_desc, law_cat_cd, arrest_boro, ...)
```

Przykładowy rowkey:

```
202512_<geohash7>_<uuid>
```

Dzięki tak zdefiniowanej strukturze:

- wszystkie surowe pliki trafiają bezpośrednio z NiFi do HDFS (brak duplikacji na dysku VM),
- Spark i Hive mają natychmiastowy dostęp do danych bez dodatkowych kopii,
- łatwo skalować projekt w przyszłości na klaster wielowęzłowy,
- partner pracujący zdalnie widzi dokładnie tę samą strukturę po uruchomieniu środowiska lokalnego.

5 Apache NiFi - ingest i preprocessing danych

Apache NiFi pełni w projekcie rolę warstwy ingestu oraz wstępnego przetwarzania danych, odpowiedzialnej za pobieranie danych z różnych źródeł, ich walidację, czyszczenie oraz routing do odpowiednich systemów docelowych (HDFS oraz Apache Kafka). NiFi umożliwia graficzne definiowanie przepływów danych (flow), dzięki czemu proces ingestu jest czytelny, modularny i łatwy do modyfikacji.

W projekcie zaimplementowano trzy niezależne przepływy NiFi, odpowiadające trzem źródłom danych:

- dane ofert Airbnb (przetwarzanie wsadowe),
- dane policyjne z Nowego Jorku (symulowany strumień),
- dane policyjne z Wielkiej Brytanii (symulowany strumień, wejście z HDFS).

5.0.1 Tabele wymiarowe używane w projekcie

Tabele wymiarowe zostały wygenerowane w formie plików CSV przy użyciu skryptu Python i załadowane do Hive jako zewnętrzne tabele TEXTFILE. Ich celem jest zapewnienie spójnego modelowania wymiarów dla danych Airbnb oraz zdarzeń policyjnych (NYPD i UKPD). Tabele wymiarowe umożliwiają oddzielenie opisowych atrybutów od danych faktów, co prowadzi do znaczącego zmniejszenia redundancji i rozmiaru plików w warstwie przetworzonej. Dzięki temu operacje zapisu, odczytu i joinów stają się szybsze.

Główne tabele wymiarowe obejmują:

- **dim_date** – kalendarz od 2000-01-01 do 2030-12-31 z kluczami daty i atrybutami czasowymi,
- **dim_host_response_time** – kategorie czasu odpowiedzi hosta (Airbnb),
- **dim_room_type** – typy zakwaterowania (Airbnb),
- **dim_LAW_CAT_CD** – kategorie przestępstw według prawa (NYPD),
- **dim_JURISDICTION_CODE** – jurysdykcje aresztowań (NYPD),
- **dim_ARREST_BORO** – dzielnice aresztowań (NYPD),
- **dim_age_group** – grupy wiekowe sprawców (NYPD),
- **dim_outcome_category** – kategorie wyników spraw (UKPD),
- **dim_crime_type** – typy przestępstw (UKPD).

W każdej tabeli poza **dim_JURISDICTION_CODE_data** wartość klucza -1 reprezentuje brak wiedzy na temat rzeczywistej wartości pola w obserwacji.

5.1 Apache NiFi — przepływ ingestu i preprocessing danych Airbnb

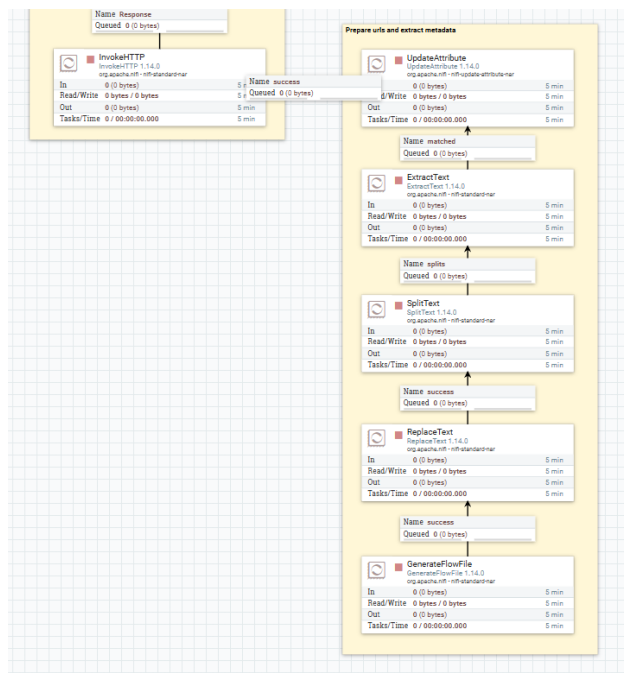
Dane ofert Airbnb są w projekcie przetwarzane w trybie wsadowym. Apache NiFi odpowiada za pobranie plików źródłowych z serwisu Inside Airbnb, wstępne czyszczenie danych, standaryzację metadanych oraz zapis przetworzonych rekordów do rozproszonego systemu plików HDFS. Cały przepływ został podzielony na kilka logicznych etapów, co zwiększa czytelność oraz ułatwia dalszą rozbudowę pipeline'u.

Przygotowanie adresów URL i ekstrakcja metadanych

Pierwszy etap przepływu odpowiada za wygenerowanie adresu URL prowadzącego do pliku `listings.csv.gz` oraz wyodrębnienie metadanych zawartych w adresie, takich jak nazwa miasta oraz data snapshotu danych. Informacje te są następnie wykorzystywane do:

- standaryzacji nazw plików wyjściowych,
- budowy docelowych ścieżek zapisu danych w HDFS,

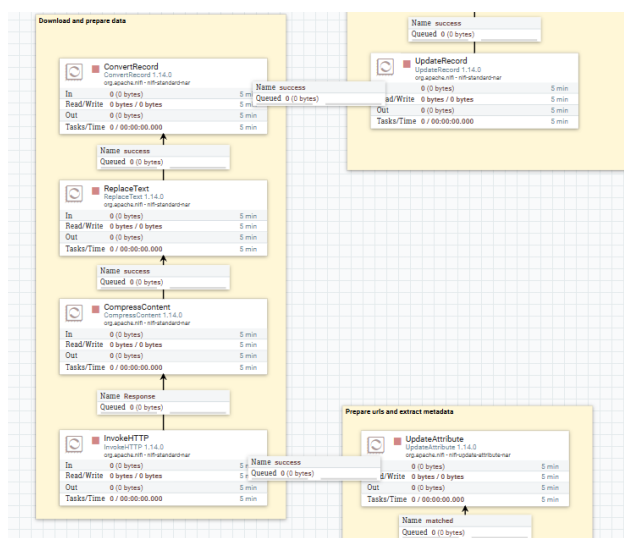
- dalszego sterowania przepływem danych.



Rysunek 1: Apache NiFi — przygotowanie adresów URL i ekstrakcja metadanych dla danych Airbnb

Pobieranie i wstępne przygotowanie danych

Na kolejnym etapie dane są pobierane z serwisu Inside Airbnb przy użyciu protokołu HTTP. Pliki źródłowe są skompresowane w formacie GZIP, dlatego przepływ obejmuje również ich rozpakowanie. Dodatkowo wykonywane są operacje naprawcze struktury pliku CSV, w szczególności korekta nieprawidłowych zakończeń linii, które mogłyby powodować błędy w dalszym przetwarzaniu. Finalnie dane parsowane są przez procesor ConvertRecord z wykorzystaniem AvroRecordSetWriter jako Record Writer.

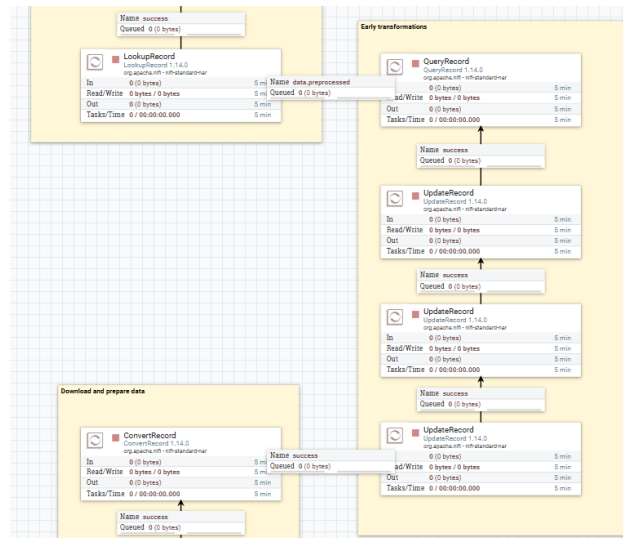


Rysunek 2: Apache NiFi — pobieranie i wstępne przygotowanie danych Airbnb

Wczesne transformacje rekordów

Po wczytaniu danych do postaci rekordowej wykonywane są wczesne transformacje, których celem jest ujednolichenie i oczyszczenie danych. Na tym etapie realizowane są m.in.:

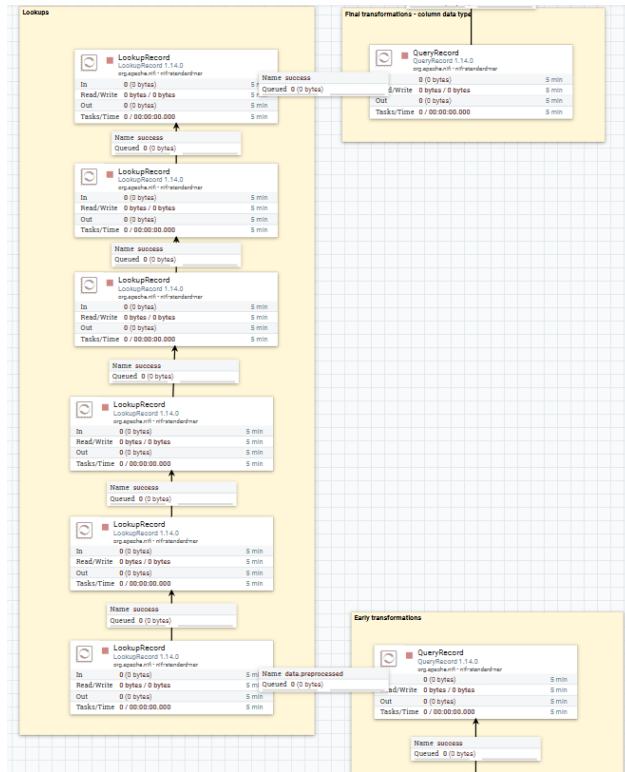
- czyszczenie pól liczbowych (ceny, procenty) z symboli walut i separatorów,
- normalizacja pól tekstowych (np. typ pokoju, opisy),
- parsowanie i standaryzacja pól dat,
- selekcja atrybutów istotnych z punktu widzenia dalszych analiz.



Rysunek 3: Apache NiFi — wczesne transformacje rekordów danych Airbnb

Mapowania słownikowe (lookups)

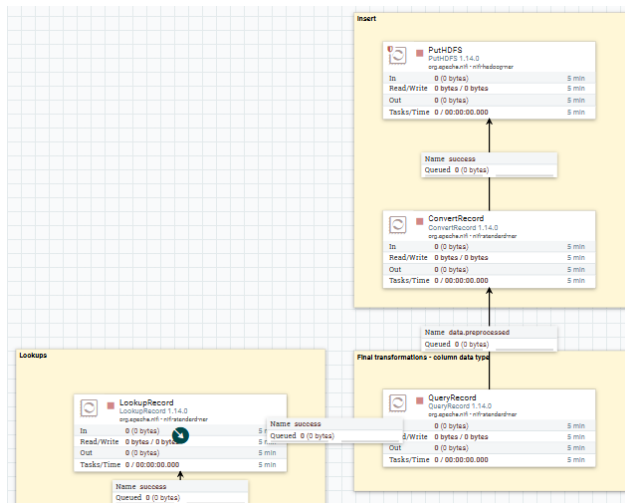
W kolejnym etapie wykorzystywane są mechanizmy mapowań słownikowych (*lookups*), które pozwalają na ujednolichenie wartości kategorycznych oraz kodowanie wybranych atrybutów. Zapewnia to spójność danych pomiędzy różnymi zbiorami oraz ułatwia ich późniejsze łączenie i agregowanie w Apache Spark.



Rysunek 4: Apache NiFi — mapowania słownikowe w danych Airbnb

Finalne przekształcenia i zapis do HDFS

Ostatni etap przepływu obejmuje finalne rzutowanie typów danych oraz zapis przetworzonych rekordów do rozproszonego systemu plików HDFS w warstwie *processed*. Dane zapisane na tym etapie stanowią bezpośrednie wejście dla zadań przetwarzania wsadowego realizowanych w Apache Spark.



Rysunek 5: Apache NiFi — finalne przekształcenia i zapis danych Airbnb do HDFS

5.1.1 Struktura danych ładujących w Hive i Kafka

Dane Airbnb po przetworzeniu w Apache NiFi są zapisywane w formacie Avro, a następnie konwertowane do Parquet (dla warstwy analitycznej w Hive).

Schemat Avro (nifiRecord) zawiera następujące pola z typami danych:

- `id` - `string` - unikalny identyfikator oferty Airbnb
- `scrape_id` - `string` - identyfikator scrapowania danych
- `last_scraped_id` - `int` - klucz wymiarowy daty ostatniego scrapowania
- `name` - `string` - nazwa oferty
- `description` - `string` - szczegółowy opis oferty
- `neighborhood_overview` - `string` - opis okolicy przez gospodarza
- `host_id` - `string` - unikalny identyfikator gospodarza
- `host_name` - `string` - imię lub nazwa gospodarza
- `host_since_id` - `int` - klucz wymiarowy daty rejestracji gospodarza
- `host_location_country` - `string` - kraj podany przez gospodarza
- `host_location_city` - `string` - miasto podane przez gospodarza
- `host_about` - `string` - opis gospodarza
- `host_response_time` - `int` - klucz wymiarowy czasu odpowiedzi gospodarza
- `host_response_rate` - `float` - procent akceptowanych rezerwacji
- `host_acceptance_rate` - `float` - procent odpowiedzi na zapytania
- `host_is_superhost` - `boolean` - status Superhost (true/false)
- `host_listings_count` - `int` - liczba ofert gospodarza
- `host_total_listings_count` - `int` - całkowita liczba ofert (wg Airbnb)
- `host_has_profile_pic` - `boolean` - czy gospodarz ma zdjęcie profilowe
- `host_identity_verified` - `boolean` - czy tożsamość gospodarza zweryfikowana
- `neighbourhood_cleansed` - `string` - oczyszczona nazwa dzielnicy
- `latitude` - `float` - szerokość geograficzna
- `longitude` - `float` - długość geograficzna
- `property_type` - `string` - typ nieruchomości
- `room_type` - `int` - klucz wymiarowy typu pokoju
- `accommodates` - `int` - maksymalna liczba gości
- `bathrooms` - `float` - liczba łazienek
- `bedrooms` - `float` - liczba sypialni
- `beds` - `int` - liczba łóżek
- `price` - `float` - cena za dobę

- `minimum_nights` - `int` - minimalna liczba nocy
- `maximum_nights` - `int` - maksymalna liczba nocy
- `availability_30` - `int` - dostępność w ciągu 30 dni
- `availability_60` - `int` - dostępność w ciągu 60 dni
- `availability_90` - `int` - dostępność w ciągu 90 dni
- `availability_365` - `int` - dostępność w ciągu 365 dni
- `number_of_reviews` - `int` - całkowita liczba opinii
- `number_of_reviews_ltm` - `int` - liczba opinii w ostatnich 12 miesiącach
- `number_of_reviews_l30d` - `int` - liczba opinii w ostatnich 30 dniach
- `estimated_revenue_l365d` - `float` - szacowany roczny przychód
- `first_review_id` - `int` - klucz wymiarowy pierwszej opinii
- `last_review_id` - `int` - klucz wymiarowy ostatniej opinii
- `review_scores_rating` - `float` - ogólna ocena
- `review_scores_accuracy` - `float` - dokładność opisu
- `review_scores_cleanliness` - `float` - czystość
- `review_scores_checkin` - `float` - łatwość zameldowania
- `review_scores_communication` - `float` - komunikacja z gospodarzem
- `review_scores_location` - `float` - lokalizacja
- `review_scores_value` - `float` - stosunek jakości do ceny
- `reviews_per_month` - `float` - średnia liczba opinii na miesiąc
- `has_verifications_email` - `boolean` - czy e-mail zweryfikowany
- `has_verifications_phone` - `boolean` - czy telefon zweryfikowany
- `has_verifications_work_email` - `boolean` - czy służbowy e-mail zweryfikowany
- `amenities_has_wifi` - `boolean` - czy Wi-Fi w udogodnieniach
- `amenities_has_smoke_alarm` - `boolean` - czy czujka dymu
- `amenities_has_kitchen` - `int` - liczba udogodnień kuchennych (kod)
- `amenities_has_washer` - `boolean` - czy pralka
- `amenities_has_essentials` - `boolean` - czy podstawowe udogodnienia
- `amenities_has_iron` - `boolean` - czy żelazko
- `amenities_has_hot_water` - `boolean` - czy ciepła woda
- `amenities_has_hangers` - `boolean` - czy wieszaki
- `amenities_has_cma` - `boolean` - czy klimatyzacja (cma = cooling)
- `amenities_has_heating` - `boolean` - czy ogrzewanie
- `amenities_has_tv` - `boolean` - czy telewizor
- `amenities_has_microwave` - `boolean` - czy mikrofalówka

5.2 Ingest i preprocessing danych policyjnych NYC

Dane policyjne z Nowego Jorku są pobierane bezpośrednio z API NYC Open Data (Socrata). W celu obsługi dużych wolumenów danych zastosowano mechanizm stronicowania (parametry `limit` i `offset`), generowany dynamicznie w NiFi.

Na etapie ingestu i preprocessing'u wykonywane są m.in.:

- pobranie danych z API w formacie CSV,
- czyszczenie i normalizacja pól tekstowych (opisy przestępstw, cechy demograficzne),
- ujednolicenie formatów dat oraz wyznaczenie identyfikatora daty (`date_id`),
- filtrowanie rekordów bez kluczowych informacji (brak współrzędnych, daty lub opisu),
- mapowanie wybranych pól kategorycznych na kody numeryczne przy użyciu lookupów,
- zaokrąglenie współrzędnych geograficznych do stałej precyzji.

Przetworzone dane są następnie kierowane dwutorowo:

- do Apache Kafka (topic `police-stream`) w celu symulacji strumieniowego napływu zdarzeń,
- do HDFS w warstwie *processed* jako dane referencyjne.

5.2.1 Struktura danych NYPD łądujących w Hive i Kafka

Dane NYPD po przetworzeniu w Apache NiFi są zapisywane w formacie Avro, a następnie konwertowane do Parquet (dla warstwy analitycznej w Hive) oraz przesyłane do Apache Kafka w celu symulacji strumieniowania.

Schemat Avro (NYPDArrestRecord) zawiera następujące pola z typami danych:

- `arrest_key` – `long` – unikalny identyfikator aresztowania (persistent ID)
- `arrest_date_id` – `int` – klucz wymiarowy daty aresztowania (YYYYMMDD)
- `pd_cd` – `int` – trzycyfrowy kod klasyfikacji wewnętrznej (bardziej szczegółowy niż KY_CD)
- `pd_desc` – `string` – opis klasyfikacji wewnętrznej odpowiadający PD_CD
- `ky_cd` – `int` – trzycyfrowy kod klasyfikacji ogólnej (kategoria przestępstwa)
- `ky_desc` – `string` – opis ogólnej kategorii przestępstwa odpowiadający KY_CD
- `law_cat_cd` – `int` – poziom przestępstwa (felony, misdemeanor, violation)
- `arrest_boro` – `int` – kod dzielnicy aresztowania (Bronx, Brooklyn, Manhattan, Queens, Staten Island)
- `arrest_precinct` – `int` – numer komisariatu, w którym doszło do aresztowania
- `jurisdiction_code` – `int` – kod jurysdykcji odpowiedzialnej za aresztowanie (NYPD Patrol, Transit, Housing lub inne)
- `age_group` – `int` – kod grupy wiekowej sprawcy
- `perp_sex` – `string` – płeć sprawcy (M/F/U)
- `latitude` – `double` – szerokość geograficzna zdarzenia
- `longitude` – `double` – długość geograficzna zdarzenia
- `city_code` – `string` – kod miasta (NYC)
- `source` – `string` – źródło danych (np. NYC Open Data)

5.3 Ingest i preprocessing danych policyjnych UK

Dane policyjne z Wielkiej Brytanii są przetwarzane w odmienny sposób niż dane NYC. W tym przypadku nie korzystano z API, lecz z lokalnych plików CSV, które w pierwszym kroku są ładowane do HDFS w warstwie *raw*. Następnie NiFi pobiera te dane bezpośrednio z HDFS i wykonuje preprocessing.

Etap ten obejmuje:

- czyszczenie opisów lokalizacji (usunięcie prefiksów typu *“On or near”*),
- uzupełnianie brakującej informacji o dniu miesiąca w polu *Month* w celu utworzenia pełnej daty,
- filtrowanie oraz selekcję kluczowych pól analitycznych,
- mapowanie typów przestępstw i wyników postępowań przy użyciu tabel wymiarów,
- standaryzację współrzędnych geograficznych.

Podobnie jak w przypadku danych NYC, dane UK są kierowane do Apache Kafka w celu symulacji strumienia oraz zapisywane w HDFS jako dane przetworzone.

5.3.1 Struktura danych UKPD ładujących w Hive i Kafka

Dane policyjne z Wielkiej Brytanii (*data.police.uk*) po przetworzeniu w Apache NiFi są zapisywane w formacie Avro, a następnie konwertowane do Parquet (dla warstwy analitycznej w Hive) oraz przesyłane do Apache Kafka w celu symulacji strumieniowania.

Schemat Avro (*LondonPoliceRecord*) zawiera następujące pola z typami danych:

- *crime_id* – *string* – unikalny identyfikator zdarzenia przestępczego
- *date_id* – *int* – klucz wymiarowy miesiąca zdarzenia (YYYYMM)
- *longitude* – *double* – długość geograficzna zdarzenia
- *latitude* – *double* – szerokość geograficzna zdarzenia
- *location* – *string* – opis miejsca zdarzenia (np. ulica, punkt orientacyjny)
- *lsoa_code* – *string* – kod Lower Layer Super Output Area (jednostka statystyczna)
- *lsoa_name* – *string* – nazwa obszaru LSOA
- *crime_type* – *int* – klucz wymiarowy typu przestępstwa
- *last_outcome_category* – *int* – klucz wymiarowy kategorii ostatniego wyniku sprawy
- *city_code* – *string* – kod miasta lub regionu policyjnego
- *source* – *string* – źródło danych

5.4 Rola Apache NiFi w architekturze

Apache NiFi umożliwił oddzielenie logiki ingestu i wstępnego czyszczenia danych od właściwego przetwarzania analitycznego realizowanego w Apache Spark. Dzięki temu:

- dane trafiają do Spark już w ujednoliconej i oczyszczonej postaci,
- możliwe jest niezależne sterowanie trybem batchowym i strumieniowym,
- pipeline jest modularny i łatwy do rozbudowy o kolejne źródła danych.

Zastosowane przepływy NiFi stanowią stabilną warstwę wejściową całego systemu Big Data i zapewniają spójność danych wykorzystywanych w dalszych etapach analizy.

6 Apache Spark - Przetwarzanie batchowe (Spark Batch)

Przetwarzanie wsadowe danych w projekcie zostało zrealizowane z wykorzystaniem Apache Spark i stanowi kluczowy element pipeline'u analitycznego. Celem etapu batchowego jest przygotowanie spójnych, oczyszczonych i zagregowanych zbiorów danych, umożliwiających analizę zależności pomiędzy rozmieszczeniem ofert Airbnb a poziomem przestępczości w analizowanych miastach.

Cały proces batchowy został podzielony na cztery logiczne etapy, zaimplementowane jako osobne notebooki Spark, z których każdy zapisuje wyniki do HDFS w formacie Parquet.

6.1 Wspólny klucz przestrzenny (geohash)

W celu efektywnego łączenia danych geolokalizowanych zastosowano klucz przestrzenny `geohash5`, obliczany na podstawie współrzędnych geograficznych. Zastosowanie geohash o precyzji 5 znaków umożliwia agregację danych w jednolitej siatce przestrzennej oraz znacząco upraszcza operacje join w porównaniu do kosztownych joinów poligonowych.

Listing 1: Wyznaczanie klucza przestrzennego geohash5

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType
import geohash2

geohash_udf = udf(
    lambda lat, lon: geohash2.encode(lat, lon, precision=5),
    StringType()
)

df = df.withColumn(
    "geohash5",
    geohash_udf(col("latitude_d"), col("longitude_d"))
)
```

Dodatkowo wszystkie rekordy są oznaczane kodem miasta (`city_code`), co umożliwia prowadzenie analiz porównawczych w ramach jednego, ujednoliconego modelu danych.

6.2 Notebook 1: Czyszczenie danych Airbnb

Pierwszy etap batchowy obejmuje czyszczenie oraz standaryzację danych ofert Airbnb. Notebook `1_Airbnb_cleaned+geohash.ipynb` wczytuje dane z HDFS, wykonuje walidację pól geograficznych oraz przygotowuje dane do dalszych analiz.

- Wejście: `/processed/airbnb_listings`
- Wyjście: `/processed/airbnb_cleaned`

Kluczowym elementem tego etapu jest czyszczenie współrzędnych geograficznych oraz usunięcie rekordów nieprawidłowych:

Listing 2: Czyszczenie współrzędnych geograficznych ofert Airbnb

```
airbnb = airbnb \
    .withColumn("latitude_d", col("latitude").cast("double")) \
    .withColumn("longitude_d", col("longitude").cast("double")) \
    .filter(col("latitude_d").between(-90, 90)) \
    .filter(col("longitude_d").between(-180, 180))
```

Dodatkowo parsowana jest cena noclegu do postaci numerycznej, usuwane są duplikaty po identyfikatorze oferty oraz wybierany jest zestaw kluczowych kolumn analitycznych. Wynikiem etapu jest spójny zbiór ofert Airbnb zapisany w formacie Parquet.

6.3 Notebook 2: Unifikacja danych policyjnych

Drugi notebook odpowiada za ujednolicenie danych policyjnych pochodzących z dwóch niezależnych źródeł: `data.police.uk` oraz NYC Open Data. Dane te różnią się schematem i nazewnictwem kolumn, dlatego zostały przekształcone do wspólnego modelu.

- **Wejście:** dane UK (Parquet) oraz NYC (Parquet)
- **Wyjście:** `/processed/police_unified`

Przykład standaryzacji schematu danych policyjnych z Wielkiej Brytanii:

Listing 3: Standaryzacja schematu danych policyjnych (UK)

```
police_uk = police_uk.select(
    col("Crime ID").alias("event_id"),
    col("Month").alias("date_id"),
    lit("LON").alias("city_code"),
    lit("UK_POLICE").alias("source"),
    col("Latitude").cast("double").alias("latitude_d"),
    col("Longitude").cast("double").alias("longitude_d"),
    col("Crime type").alias("crime_type_id")
)
```

Po standaryzacji dane z obu źródeł są łączone operacją `unionByName`, co pozwala na zachowanie spójnego schematu mimo różnic w danych wejściowych.

6.4 Notebook 3: Join danych Airbnb z przestępczością

Trzeci etap batchowy realizuje połączenie danych Airbnb z danymi o przestępczości zagregowanymi w siatce geohash. Najpierw obliczane są agregaty przestępstw na poziomie (`city_code`, `geohash5`):

Listing 4: Agregacja zdarzeń policyjnych w siatce geohash

```
police_geo_agg = police.groupBy("city_code", "geohash5").agg(
    count("*").alias("crime_events_total"),
    countDistinct("event_id").alias("crime_events_distinct"),
    countDistinct("crime_type_id").alias("crime_types_distinct")
)
```

Następnie wykonywany jest lewostronny join danych Airbnb z agregatami policyjnymi po `city_code` i `geohash5`:

Listing 5: Join danych Airbnb z agregatami przestępczości

```
listings_with_stats = airbnb.alias("a").join(
    police_geo_agg.alias("p"),
    on=[
        col("a.city_code") == col("p.city_code"),
        col("a.geohash5") == col("p.geohash5")
    ],
    how="left"
)
```

Zastosowanie *left join* gwarantuje zachowanie wszystkich ofert Airbnb w zbiorze wynikowym, nawet jeśli dla danego obszaru nie występują zdarzenia policyjne. Braki w licznikach przestępstw są uzupełniane zerami, a następnie wyznaczany jest indeks bezpieczeństwa:

Listing 6: Wyznaczenie indeksu bezpieczeństwa

```
listings_with_stats = listings_with_stats.fillna({
    "crime_events_total": 0})
).withColumn(
    "safety_index",
    1 / (1 + col("crime_events_total"))
)
```

6.5 Notebook 4: Statystyki dzielnic i porównania miast

Ostatni notebook buduje finalne tabele raportowe, wykorzystywane w zapytaniach Hive oraz warstwie wizualizacji. Dane agregowane są zarówno na poziomie dzielnic, jak i całych miast.

Listing 7: Agregacje statystyk na poziomie dzielnic

```
neighbourhood_stats = listings.groupBy(
    "city_code", "neighbourhood_cleansed"
).agg(
    count("*").alias("listings_total"),
    avg("price_num").alias("avg_price"),
    avg("crime_events_total").alias("avg_crimes"),
    avg("safety_index").alias("avg_safety")
)
```

Dodatkowo wyznaczane są korelacje pomiędzy ceną, ocenami oraz poziomem przestępczości, które następnie dołączane są do tabel porównawczych miast.

6.6 Podsumowanie etapu batchowego

Efektom przetwarzania wsadowego jest zestaw spójnych i ustandaryzowanych zbiorów danych w formacie Parquet, gotowych do dalszej analizy i raportowania. Zaprojektowany pipeline Spark Batch zapewnia powtarzalność przetwarzania, wysoką jakość danych oraz możliwość łatwej rozbudowy projektu o kolejne miasta lub okresy czasowe.

7 Apache Hive (SQL interface)

Apache Hive pełni w projekcie rolę warstwy analityczno-raportowej, zapewniając dostęp SQL do danych przygotowanych w poprzednich etapach przetwarzania wsadowego w Apache Spark. Dane wejściowe dla Hive są przechowywane w HDFS w formacie Parquet i reprezentują zarówno dane po czyszczeniu (*processed*), jak i gotowe agregaty analityczne (*analytical*).

Zastosowano podejście oparte na **tabelach zewnętrznych (external tables)**, w którym Hive przechowuje wyłącznie metadane (schemat), natomiast dane fizycznie pozostają w HDFS. Rozwiązanie to jest zgodne z architekturą Data Lake oraz umożliwia ponowne przetwarzanie danych w Spark bez konieczności redefiniowania warstwy SQL.

7.1 Warstwa tabel zewnętrznych

W bazie danych `bigdata` utworzono zestaw tabel zewnętrznych mapujących katalogi Parquet w HDFS. Najważniejsze tabele wykorzystywane w analizach to:

- `airbnb_cleaned` - dane ofert Airbnb po czyszczeniu i standaryzacji,
- `police_unified` - ujednolicone dane policyjne z Wielkiej Brytanii i Nowego Jorku,
- `listings_with_police_stats` - wynik joinu ofert Airbnb z agregatami przestępczości,
- `neighbourhood_stats` - statystyki zagregowane na poziomie dzielnic,
- `city_comparisons` - porównania międzymiastowe wraz z korelacjami.

Poniżej przedstawiono skrócony przykład definicji tabeli zewnętrznej Hive nad danymi Parquet:

Listing 8: Przykład tabeli zewnętrznej Hive

```
CREATE EXTERNAL TABLE airbnb_cleaned (  
  id STRING,  
  city_code STRING,  
  neighbourhood_cleaned STRING,  
  latitude_d DOUBLE,  
  longitude_d DOUBLE,  
  geohash5 STRING,  
  price_num DOUBLE  
)  
STORED AS PARQUET  
LOCATION '/user/vagrant/bigdata/processed/airbnb_cleaned';
```

W definicjach tabel zadbano o zgodność typów danych z wynikami Apache Spark, w szczególności pola pochodzące z agregacji (`count`) zostały zdefiniowane jako `BIGINT`.

7.2 Pytania analityczne i wyniki

Warstwa Apache Hive została wykorzystana do odpowiedzi na kluczowe pytania analityczne dotyczące zależności pomiędzy rozmieszczeniem ofert Airbnb a poziomem przestępczości w ich otoczeniu. Analiza opiera się na danych przetworzonych w Apache Spark i zapisanych w warstwie *analytical*, co umożliwiło wykonywanie zapytań SQL nad ujednoliconym i przygotowanym zbiorem danych.

Należy zaznaczyć, że analiza prowadzona jest na zbiorach o różnej skali, co wynika z ograniczeń wydajnościowych środowiska jednowęzłowego. W szczególności:

- dla Londynu oraz Nowego Jorku wykorzystano po 200 ofert Airbnb (próba losowa),
- dane policyjne dla Nowego Jorku obejmują około 400 zdarzeń,
- dane policyjne dla Wielkiej Brytanii (UKPD) obejmują około 79 tysięcy zdarzeń.

Różnice te mają istotny wpływ na interpretację wyników i zostały uwzględnione w dalszych analizach.

Ceny noclegów a poziom przestępczości

Tabela 1: Średnia i mediana ceny noclegu w podziale na miasta

Miasto	Średnia cena	Mediana ceny
LON	153.23	105.00
NYC	210.05	149.80

W Nowym Jorku zarówno średnia, jak i mediana ceny noclegu są wyraźnie wyższe niż w Londynie. Różnica ta może wynikać z odmiennej struktury rynku nieruchomości, wyższego progu cenowego w centralnych dzielnicach Nowego Jorku oraz większego udziału ofert premium w analizowanej próbie danych.

Rozkład ofert według intensywności przestępczości

Tabela 2: Rozkład ofert Airbnb według kategorii przestępczości

Miasto	crime_bucket	Liczba ofert
LON	21+	200
NYC	1-5	30
NYC	6-20	101
NYC	21+	69

Kategorie `crime_bucket` zostały zdefiniowane jako przedziały liczby zdarzeń policyjnych przypisanych do pojedynczej oferty Airbnb na podstawie jej lokalizacji (geohash):

- 0 - brak zdarzeń,
- 1-5 - niska intensywność przestępczości,
- 6-20 - umiarkowana intensywność,
- 21+ - wysoka intensywność przestępczości.

W przypadku Londynu wszystkie analizowane oferty znajdują się w kategorii 21+. Jest to bezpośredni efekt bardzo dużej liczby zdarzeń policyjnych w danych UKPD (ok. 79 tys.) oraz wysokiej gęstości raportowania przestępstw w centralnych obszarach miasta. W Nowym Jorku rozkład ofert pomiędzy kategoriami jest bardziej zróżnicowany, co wynika z mniejszej liczby zdarzeń policyjnych oraz innej skali raportowania danych.

Bezpieczeństwo w otoczeniu ofert Airbnb

Tabela 3: Średnia i mediana przestępczości oraz wskaźnika bezpieczeństwa

Miasto	Avg crimes	Median crimes	Avg safety	Median safety
LON	1954.31	1739.73	0.00093	0.00057
NYC	17.11	11.93	0.09293	0.05822

Wartości przedstawione w tabeli odnoszą się do liczby zdarzeń policyjnych **przypisanych do pojedynczej oferty Airbnb** na podstawie jej lokalizacji. W szczególności *Avg crimes* oraz *Median crimes* oznaczają średnią oraz medianę liczby zdarzeń policyjnych występujących w bezpośrednim otoczeniu pojedynczej oferty, a nie całkowitą liczbę przestępstw w danym mieście.

Bardzo wysoka wartość średniej liczby zdarzeń w Londynie wynika z połączenia dużej liczby zgłoszeń policyjnych oraz wysokiej gęstości przestrzennej zdarzeń w danych UKPD. Dla Nowego Jorku wartości te są znacznie niższe i bardziej porównywalne z intuicyjnym postrzeganiem skali przestępczości w otoczeniu pojedynczej oferty.

Zależności pomiędzy ceną, oceną a przestępczością

Tabela 4: Korelacje pomiędzy wybranymi zmiennymi

Miasto	Cena vs przestępczość	Ocena vs przestępczość	Cena vs bezpieczeństwo
LON	0.173	0.160	-0.078
NYC	-0.099	0.053	0.084

Współczynniki korelacji opisują siłę liniowej zależności pomiędzy analizowanymi zmiennymi. Uzyskane wartości wskazują na ogólnie słabe zależności pomiędzy poziomem przestępczości a cenami oraz ocenami ofert Airbnb, jednak ich znaki i wielkości pozwalają na jakościową interpretację trendów.

W przypadku Londynu dodatnia korelacja pomiędzy ceną a poziomem przestępczości (*Cena vs przestępczość*) sugeruje, że droższe oferty częściej występują w obszarach o wyższej intensywności zdarzeń policyjnych, co może wynikać z koncentracji ofert premium w centralnych, gęsto zabudowanych dzielnicach miasta. Jednocześnie ujemna korelacja pomiędzy ceną a wskaźnikiem bezpieczeństwa (*Cena vs bezpieczeństwo*) wskazuje, że wraz ze wzrostem ceny obserwowany jest spadek wartości wskaźnika bezpieczeństwa.

Dla Nowego Jorku zależności te mają odmienny charakter. Ujemna korelacja pomiędzy ceną a poziomem przestępczości sugeruje, że droższe oferty są częściej zlokalizowane w bezpieczniejszych obszarach miasta. Dodatnia korelacja pomiędzy ceną a wskaźnikiem bezpieczeństwa potwierdza ten trend, wskazując na wyraźniejsze powiązanie cen z bezpieczeństwem otoczenia.

Korelacje pomiędzy oceną oferty a poziomem przestępczości są w obu miastach bliskie zeru, co sugeruje, że subiektywna ocena noclegu przez użytkowników Airbnb jest w niewielkim stopniu powiązana z poziomem przestępczości w jego otoczeniu.

7.3 Podsumowanie

Apache Hive umożliwił wygodne wykonywanie zapytań SQL nad danymi przetwarzanymi w pipeline Big Data oraz przygotowanie czytelnych zestawień raportowych. Zastosowanie tabel zewnętrznych nad plikami Parquet pozwoliło na oddzielenie warstwy przetwarzania od warstwy analitycznej i stworzenie elastycznego środowiska do dalszych analiz i wizualizacji.

8 Wizualizacja wyników

Ostatnim etapem projektu była wizualizacja wyników analiz przestrzennych, której celem było intuicyjne przedstawienie zależności pomiędzy rozmieszczeniem ofert Airbnb a intensywnością przestępczości w analizowanych miastach. Wizualizacje zostały przygotowane na podstawie danych przetworzonych w Apache Spark i zapisanych w warstwie *analytical*, a następnie przedstawione w formie interaktywnych map.

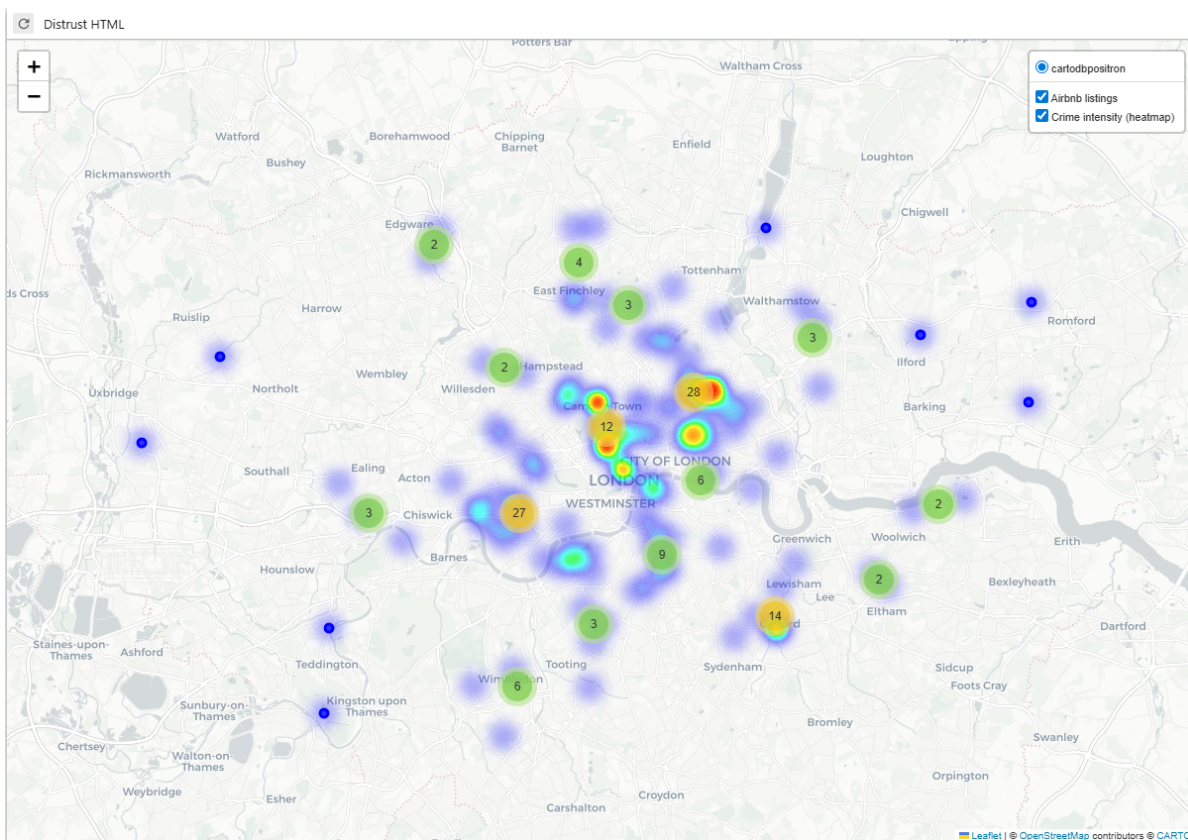
W raporcie zaprezentowano zrzuty ekranu (screenshots) wybranych wizualizacji, w szczególności map typu heatmap oraz choropleth map, które pozwalają na szybkie zidentyfikowanie obszarów o wysokiej koncentracji zdarzeń.

8.1 Heatmapa intensywności przestępczości i ofert Airbnb - Londyn

Pierwsza wizualizacja przedstawia mapę Londynu z nałożoną heatmapą intensywności przestępczości. Kolory o większej intensywności (żółty, czerwony) wskazują obszary o wysokiej liczbie zdarzeń policyjnych, natomiast chłodniejsze kolory oznaczają niższą aktywność.

Dodatkowo na mapie zaznaczono okręgi z wartościami liczbowymi, które reprezentują liczbę ofert Airbnb zlokalizowanych w danym obszarze. Pozwala to na jednoczesną analizę koncentracji ofert wynajmu krótkoterminowego oraz intensywności przestępczości.

Na mapie widoczna jest wyraźna koncentracja zdarzeń w centralnych dzielnicach miasta, takich jak Camden, Westminster oraz okolice City of London. Obszary peryferyjne charakteryzują się znacznie niższą intensywnością zgłoszeń.



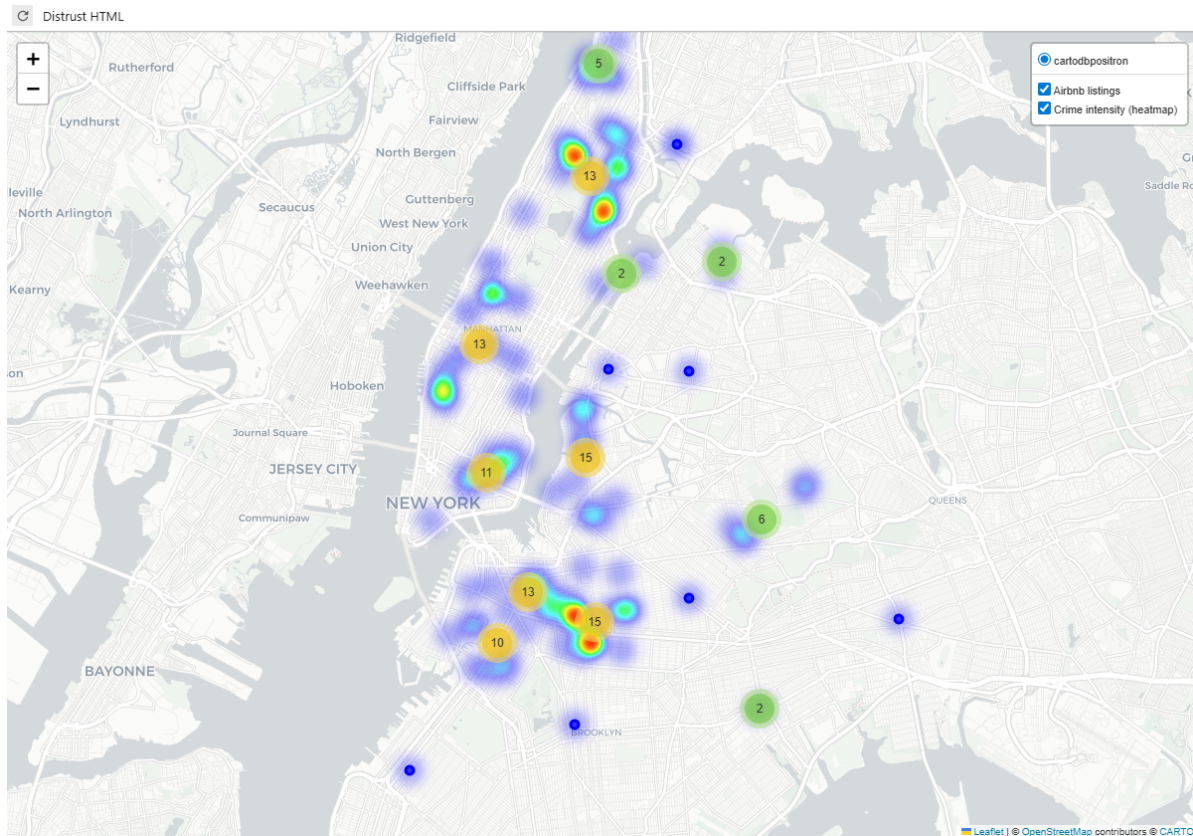
Rysunek 6: Heatmapa intensywności przestępczości w Londynie

8.2 Heatmapa intensywności przestępczości i ofert Airbnb - Nowy Jork

Kolejna wizualizacja przedstawia mapę Nowego Jorku, na której jednocześnie zaprezentowano:

- heatmapę intensywności przestępczości,
- lokalizację ofert Airbnb (punkty).

Takie połączenie umożliwia bezpośrednie porównanie rozmieszczenia ofert wynajmu krótkoterminowego z poziomem aktywności policyjnej. Widoczna jest silna koncentracja zarówno ofert Airbnb, jak i zdarzeń policyjnych na Manhattanie oraz w północnej części Brooklynu, podczas gdy obszary oddalone od centrum cechują się mniejszą gęstością obu zjawisk.



Rysunek 7: Heatmapa przestępczości oraz lokalizacji ofert Airbnb w Nowym Jorku

Znaczenie wizualizacji przestrzennych

Zastosowanie map typu heatmap pozwoliło na:

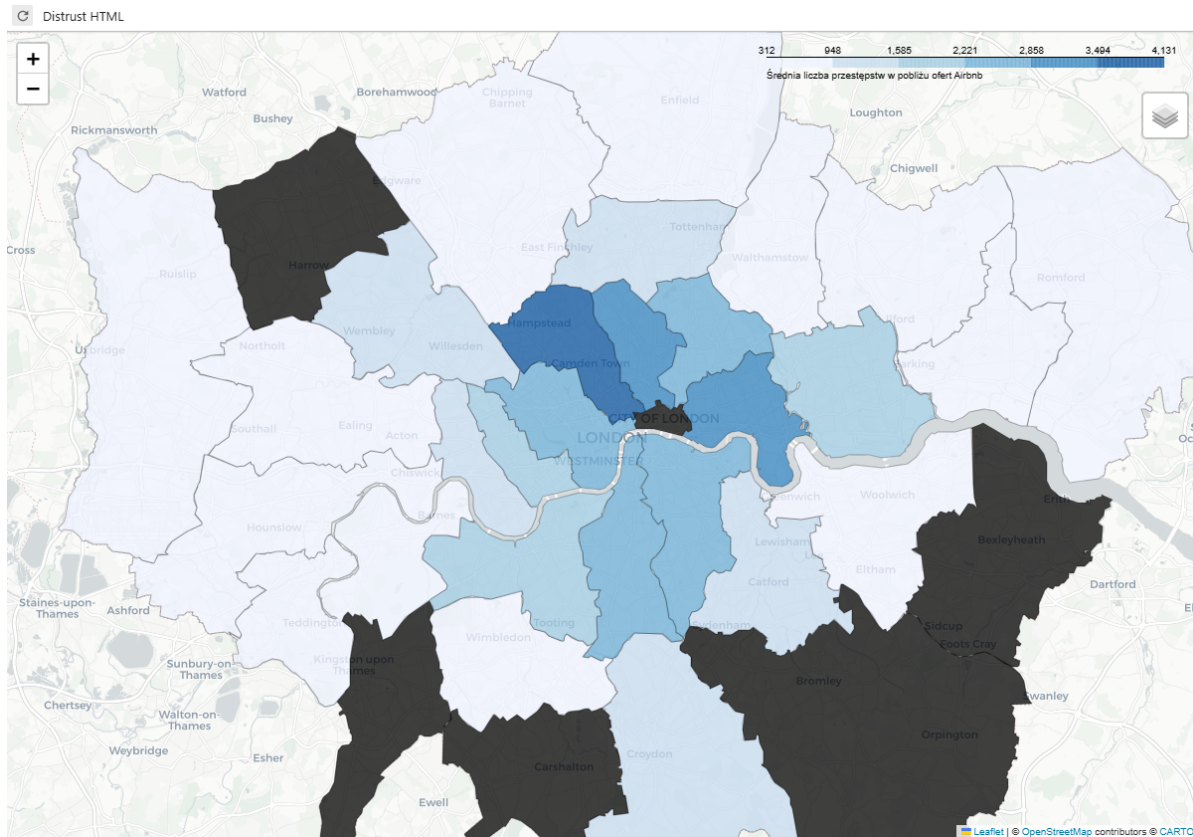
- szybkie zidentyfikowanie obszarów o wysokiej koncentracji przestępstw,
- wizualne porównanie rozmieszczenia ofert Airbnb i zdarzeń policyjnych,
- potwierdzenie wyników analiz liczbowych uzyskanych w warstwie Apache Hive.

8.3 Choropleth: średnia liczba przestępstw w pobliżu ofert Airbnb (agregacja dzielnicowa)

Oprócz map typu heatmap przygotowano mapy choropleth, które przedstawiają wyniki na poziomie dzielnic (`neighbourhood_cleansed`). Każdy poligon (dzielnica) został pokolorowany zgodnie z wartością `avg_crimes_nearby`, czyli średnią liczbą zdarzeń policyjnych przypisaną do pojedynczej oferty Airbnb w danej dzielnicy. Tego typu wizualizacja ułatwia porównania między dzielnicami i jest mniej wrażliwa na pojedyncze punkty niż klasyczna heatmapa.

Choropleth - Londyn

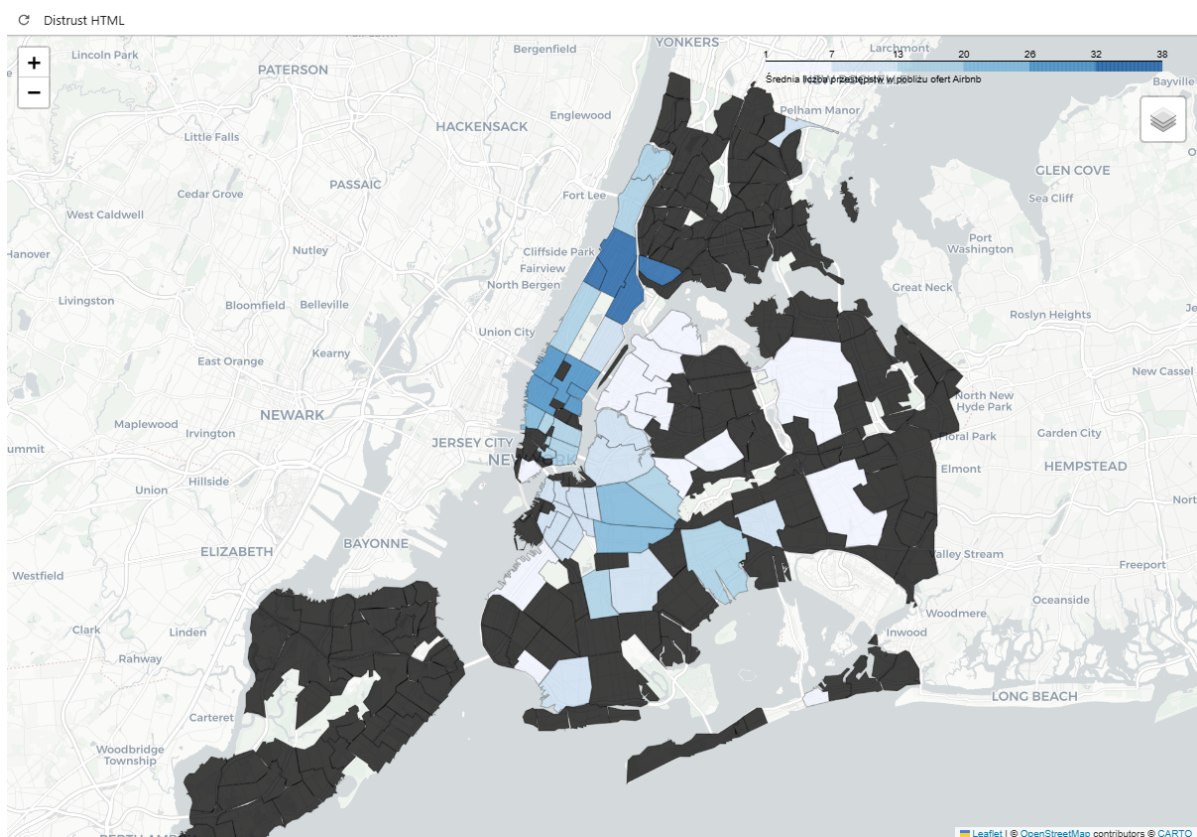
Na mapie Londynu widoczne są dzielnice o najwyższej wartości `avg_crimes_nearby` skoncentrowane w obszarach centralnych. Taki rozkład jest spójny z wcześniej zaprezentowaną heatmapą, gdzie najwyższa intensywność zdarzeń była obserwowana w centrum miasta. Część dzielnic może pozostać bez kolorowania (np. ciemne obszary), co wynika z braku danych dla danej dzielnicy w analizowanej próbie.



Rysunek 8: Choropleth Londyn: średnia liczba przestępstw w pobliżu ofert Airbnb (`avg_crimes_nearby`)

Choropleth - Nowy Jork

W przypadku Nowego Jorku choropleth pokazuje wyraźne zróżnicowanie przestrzenne wartości `avg_crimes_nearby`. Najwyższe wartości obserwowane są przede wszystkim w wybranych obszarach Manhattanu oraz w częściach Brooklynu, co jest zgodne z koncentracją punktów na mapie heatmap.



Rysunek 9: Choropleth Nowy Jork: średnia liczba przestępstw w pobliżu ofert Airbnb (`avg_crimes_nearby`)

8.4 Podsumowanie

Wizualizacje stanowią uzupełnienie analiz statystycznych i tabelarycznych, umożliwiając bardziej intuicyjną interpretację danych oraz wspierając wnioski dotyczące zależności pomiędzy intensywnością wynajmu krótkoterminowego a bezpieczeństwem publicznym w analizowanych miastach.

9 Testy

9.1 Testy funkcjonalne pipeline'u Spark

W celu weryfikacji poprawności działania przetwarzania wsadowego (Spark Batch) przygotowano test funkcjonalny uruchamiany poleceniem `spark-submit`. Test sprawdza kluczowe działania pipeline'u (Parquet w HDFS): `airbnb_cleaned`, `police_unified` oraz wynik joinu `listings_with_police_stats`. Weryfikacja obejmuje spójność schematów, podstawową jakość danych (NULL-e, zakresy współrzędnych, długość geohash) oraz poprawność logiki joinu i metryk.

Listing 9: Uruchomienie testu pipeline'u Spark

```
spark-submit test_spark_pipeline.py
```

Tabela 5: Podsumowanie testów funkcjonalnych pipeline'u Spark

ID testu	Zakres weryfikacji	Wynik
T01	Odczyt danych Parquet z HDFS: <code>airbnb_cleaned</code> , <code>police_unified</code> , <code>listings_with_police_stats</code>	PASS
T02	Sprawdzenie obecności wymaganych kolumn (schema checks) dla wszystkich trzech zbiorów	PASS
T03	Sprawdzenie, że zbiory nie są puste: <code>airbnb_cleaned > 0</code> , <code>listings_with_police_stats > 0</code> , <code>police_unified > 0</code>	PASS
T04	Jakość danych: brak NULL-i w kluczowych polach, zakresy <code>latitude/longitude</code> , długość <code>geohash5=5</code> , poprawne wartości <code>crime_bucket</code>	PASS
T05a	Spójność joinu: liczba rekordów po joinie równa liczbie listingów (left join nie usuwa ofert)	PASS
T05b	Sprawdzenie definicji metryki: <code>safety_index</code> (maks. błąd numeryczny $\leq 10^{-9}$)	PASS
T05c	Spójność bucketów: dla <code>crime_events_total = 0</code> obowiązuje <code>crime_bucket = '0'</code>	PASS
T06	Spójność pokrycia miast: ten sam zbiór wartości <code>city_code</code> w <code>airbnb_cleaned</code> i w wyniku joinu	PASS

Wynik uruchomienia testu potwierdził poprawność kluczowych etapów pipeline'u wsadowego: poprawne wczytanie danych z HDFS, zgodność schematów, podstawową jakość danych oraz poprawną logikę joinu i wyznaczanych metryk.

```
[OK] T01: wczytanie Parquet z HDFS
[OK] airbnb_cleaned: wymagane kolumny istnieją
[OK] listings_with_police_stats: wymagane kolumny istnieją
[OK] police_unified: wymagane kolumny istnieją
[OK] T02: schema OK
[OK] airbnb_cleaned: liczba wierszy > 0 (400)
[OK] listings_with_police_stats: liczba wierszy > 0 (400)
[OK] police_unified: liczba wierszy > 0 (79538)
[OK] airbnb_cleaned: brak NULL-i w kolumnach ['id', 'city_code', 'geohash5', 'latitude_d', 'longitude_d']
[OK] airbnb_cleaned: latitude_d w zakresie [-90.0,90.0]
[OK] airbnb_cleaned: longitude_d w zakresie [-180.0,180.0]
[OK] airbnb_cleaned: geohash5 ma długość 5
[OK] listings_with_police_stats: brak NULL-i w kolumnach ['id', 'city_code', 'geohash5', 'crime_events_total', 'safety_index', 'crime_bucket']
[OK] listings_with_police_stats: geohash5 ma długość 5
[OK] listings_with_police_stats: crime_bucket ma poprawne wartości (subset {'0', '21+', '6-20', '1-5'})
[OK] police_unified: brak NULL-i w kolumnach ['event_id', 'city_code', 'source', 'geohash5', 'latitude_d', 'longitude_d']
[OK] police_unified: geohash5 ma długość 5
[OK] T04: jakość danych OK
[OK] T05a: joined rows == airbnb rows (expected for left join)
[OK] T05b: safety_index zgodny z definicją
[OK] T05c: bucket '0' dla crime_events_total==0
[OK] T05: logika joinu OK
[OK] T06: city_code spójne między airbnb i joined
```

Rysunek 10: Wynik uruchomienia testów funkcjonalnych pipeline'u Spark

9.2 Testy funkcjonalne Apache Hive (tabele zewnętrzne)

W celu weryfikacji warstwy SQL przygotowano test funkcjonalny dla Apache Hive, którego zadaniem było sprawdzenie poprawności działania tabel zewnętrznych (*external tables*) utworzonych nad danymi Parquet w HDFS. Test został zrealizowany jako skrypt `hive_functional_tests.hql`, a kryterium zaliczenia (*PASS*) stanowiło wykonanie wszystkich zapytań bez błędów typu `FAILED` lub wyjątków.

Zakres testu obejmował: dostęp do bazy danych, istnienie tabel, zgodność schematów (typów kolumn), obecność danych (liczności), zapytania typu smoke (`LIMIT`), kontrolę `NULL`-i w kluczowych kolumnach oraz poprawne działanie podstawowych agregacji.

Listing 10: Uruchomienie testu funkcjonalnego Hive

```
hive -f hive_functional_tests.hql
```

Tabela 6: Podsumowanie testów funkcjonalnych Apache Hive (external tables)

ID	Zakres weryfikacji	Wynik
T00	Połączenie z bazą bigdata	PASS
T01	Istnienie tabel: <code>airbnb_cleaned</code> , <code>police_unified</code> , <code>listings_with_police_stats</code> , <code>neighbourhood_stats</code> , <code>city_comparisons</code> , <code>police_geohash_agg</code>	PASS
T02	Sprawdzenie schematów przez <code>DESCRIBE</code> (wykrywanie błędnych typów, np. <code>BIGINT</code> dla wyników <code>count()</code>)	PASS
T03	Sprawdzenie obecności danych (liczności): <code>airbnb_cleaned=400</code> , <code>listings_with_police_stats=400</code> , <code>neighbourhood_stats=79</code> , <code>city_comparisons=2</code> , <code>police_geohash_agg=486</code>	PASS
T04	Zapytania smoke (<code>LIMIT</code>) oraz odczyt tabeli <code>city_comparisons</code> (pełny rekord porównania miast)	PASS
T05	Integralność kluczy: brak <code>NULL</code> -i w krytycznych kolumnach (<code>id</code> , <code>city_code</code> , <code>geohash5</code> oraz pola metryk w joinie)	PASS
T06	Podstawowe zapytania raportowe: liczba listingów per miasto oraz rozkład <code>crime_bucket</code>	PASS
T07	Weryfikacja typów pośrednio przez agregacje (np. <code>SUM(crime_events_total)</code>)	PASS

9.3 Testy funkcjonalne Apache HBase (warstwa NoSQL)

W celu weryfikacji warstwy NoSQL przygotowano test funkcjonalny dla Apache HBase, którego zadaniem było sprawdzenie poprawności działania bazy kolumnowej oraz integracji z warstwą strumieniowania danych. Test został zrealizowany jako skrypt `hbase_test.py` wykorzystujący bibliotekę `happybase` do komunikacji z HBase przez interfejs Thrift, a kryterium zaliczenia (*PASS*) stanowiło pomyślne wykonanie wszystkich operacji CRUD oraz weryfikacji połączenia.

Zakres testu obejmował: połączenie z serwerem Thrift, listowanie tabel, zapis pojedynczego rekordu testowego, odczyt po kluczu, skanowanie zakresu rekordów oraz usunięcie danych testowych (`cleanup`).

Listing 11: Uruchomienie testu funkcjonalnego HBase

```
python hbase_test.py
```



```

[TEST] Running HBase Functional Tests
[OK] Test 1 PASSED: HBase connection successful
[OK] Test 2 PASSED: Found tables: ['orders_denormalized', 'police_events', 'trams', 'wifi']
[OK] Test 3 PASSED: Record written
[OK] Test 4 PASSED: Record retrieved: {'common:city_code': 'NYC', 'common:date_id': '20250113', 'common:geohash': 'dr5ru', 'common:latitude': '40.7128', 'common:longitude': '-74.0060', 'metadata:test': 'true'}
Row: 20250601#gcpvj#LON#0e0d940e839e3ece19666ce15a35ee394be848358415297f9b6cb464fbb3f07c
Row: 20250601#gcpvj#LON#383313001db82fa8075449fcc850eb7f2bb12dc772b05576947b0e6aefc4287a
Row: 20250601#gcpvj#LON#3a498f16bc1ef96f0debc5b35d1a392a28ed452a6e6416cee97ec26c833a00d9
Row: 20250601#gcpvj#LON#7c3052e4b331d9aae874920c1c8ac2d6eb2118a3afb6dcae6c279e8b00b15210
Row: 20250601#gcpvj#LON#7c71da856da8ab8c273a9e400e2d835fa3c197488d2b3dce7b3b926c4b8e7d11
[OK] Test 5 PASSED: Scanned 5 records
[OK] Test 6 PASSED: Test record deleted

[RESULTS] 6/6 tests passed
26/01/13 23:21:50 INFO util.ShutdownHookManager: Shutdown hook called
26/01/13 23:21:50 INFO util.ShutdownHookManager: Deleting directory /tmp/spark-07b8c92e-212b-468e-8301-28024223f05c
vagrant@node1:~/bigdata-crimes-airbnb-project-dir$

```

Rysunek 11: Wynik wykonania testów funkcjonalnych HBase – wszystkie testy zakończone sukcesem

Tabela 7: Podsumowanie testów funkcjonalnych Apache HBase

ID	Zakres weryfikacji	Wynik
T01	Połączenie z serwerem HBase Thrift (port 9090)	PASS
T02	Listowanie tabel w HBase: wykryto tabele <code>orders_denormalized</code> , <code>police_events</code> , <code>trams</code> , <code>wifi</code> .	PASS
T03	Zapis rekordu testowego do tabeli <code>police_events</code> z kluczem <code>TEST#dr5ru#NYC#9999999</code> oraz danymi w rodzinach kolumn <code>common:*</code> i <code>metadata:*</code>	PASS
T04	Odczyt rekordu testowego po kluczu – weryfikacja zgodności zapisanych wartości (geohash, współrzędne geograficzne, kod miasta)	PASS
T05	Skanowanie zakresu 5 rekordów z tabeli <code>police_events</code> – weryfikacja struktury kluczy oraz dostępności danych historycznych	PASS
T06	Usunięcie rekordu testowego (cleanup) – weryfikacja operacji DELETE	PASS

Uwaga, obecność tabel `orders_denormalized`, `trams`, `wifi` jest wynikiem pracy na laboratoriach, jednakże dla projektu istotna jest obecność `police_events`.

Test potwierdził poprawność działania warstwy NoSQL, w tym poprawną konstrukcję kluczy kompozytowych w formacie `date_id#geohash#city_code#event_id`, działanie rodzin kolumn (`common`, `source_specific`, `metadata`) oraz dostępność danych zasilanych z warstwy strumieniowania Kafka-Spark.